

Toward Trustworthy FPGA Trojan Localization from Configuration Bitstreams

Carter Antley

University of South Carolina
Dept. of Computer Science and Engineering
Columbia, SC, USA
ctantley@email.sc.edu

Rasha Karakchi (advisor)

University of South Carolina
Dept. of Computer Science and Engineering
Columbia, SC, USA
karakchi@cec.sc.edu

Abstract

Machine learning has recently emerged as an effective approach for detecting hardware Trojans directly from FPGA configuration bitstreams. While prior work demonstrated that raw-byte machine learning can accurately identify suspicious bitstreams, deployment requires more than binary detection. Once a bitstream is flagged as malicious, a security analyst must determine which configuration region contains the Trojan to support verification and forensic analysis.

In this work, we investigate the problem of trustworthy FPGA Trojan localization from configuration bitstreams. We first evaluate a naïve localization strategy based on identifying the largest differences between reference and Trojan-embedded bitstreams. Surprisingly, our experiments show that this approach primarily identifies regions of high implementation density rather than the implanted Trojan itself. Through a systematic analysis, we demonstrate that synthesis optimization, benchmark validity, and place-and-route variability introduce implementation artifacts that overwhelm Trojan-induced changes and obscure localization. We then show that preserving placement provenance through a placement-aware verification workflow substantially reduces implementation noise, enabling reliable localization to a small number of candidate configuration windows.

Our experiments reveal that independently implemented versions of the same design differ in up to 46% of configuration frames, causing implementation artifacts to dominate naïve bitstream comparison. By preserving placement provenance, implementation noise is substantially reduced, enabling reliable localization to a small number of candidate configuration windows. Placement-aware verification reduced the search space to approximately 11 candidate windows out of 1,251 windows. These findings establish that trustworthy FPGA Trojan localization is fundamentally governed by implementation provenance rather than machine-learning accuracy alone and provide practical guidelines for deployment-time FPGA hardware security.

Keywords

FPGA Security, Hardware Trojans, Bitstream Analysis, Trojan Localization, Placement-Aware Verification, Reconfigurable Computing.

1 Motivation

Field-Programmable Gate Arrays (FPGAs) have become a critical computing platform for high-performance computing (HPC), cloud services, and edge artificial intelligence because they combine high performance with post-deployment reconfigurability [1–4]. Unlike fixed-function processors, FPGA functionality is defined by binary

configuration bitstreams that program the underlying hardware fabric. This flexibility enables rapid hardware specialization, but it also introduces a critical security boundary: maliciously modified bitstreams can embed hardware Trojans capable of leaking sensitive information, altering computation, or degrading system reliability [5–8].

Recent research has shown that machine learning can detect malicious FPGA bitstreams directly from configuration data without requiring RTL, netlists, or reverse engineering. Our previous work, BLADEI, demonstrated that statistical machine learning can accurately distinguish benign and Trojan-inserted bitstreams, enabling deployment-time screening of FPGA configurations [9–12]. However, deployment requires more than binary detection. Once a bitstream has been classified as suspicious, the operator must determine *where* the Trojan resides in order to support verification, forensic analysis, or selective rejection of the configuration.

A straightforward solution is to compare a trusted and a suspicious bitstream and attribute the largest implementation differences to the Trojan. Surprisingly, our experiments show that this assumption does not hold. Implementation artifacts introduced during synthesis, placement, and routing produce configuration changes that often dominate Trojan-induced modifications, causing naïve localization methods to identify implementation density rather than malicious logic. Consequently, trustworthy FPGA Trojan localization requires understanding and controlling implementation provenance before meaningful localization can be achieved.

This work investigates the conditions required for trustworthy FPGA Trojan localization from configuration bitstreams. Through a systematic study of implementation-generated FPGA bitstreams, we demonstrate how implementation artifacts influence localization and establish the role of placement provenance in enabling reliable region-level identification of Trojan-induced configuration changes.

2 Methodology

Figure 1 illustrates the workflow used to investigate trustworthy FPGA Trojan localization from configuration bitstreams. Rather than developing a new localization algorithm, this work examines the implementation conditions required for reliable localization by systematically analyzing the bitstream generation process and the resulting FPGA configurations.

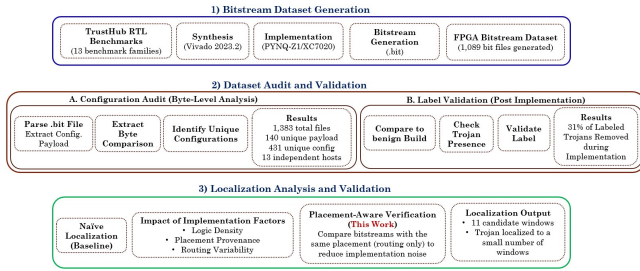


Figure 1: Workflow for investigating trustworthy FPGA Trojan localization. FPGA bitstreams are generated from RTL benchmarks, audited for configuration integrity and label validity, and analyzed using implementation-aware localization.

2.1 Bitstream Dataset Generation

The study begins with publicly available Trust-Hub RTL hardware Trojan benchmarks [13]. Since Trust-Hub provides Verilog benchmark designs rather than configuration bitstreams, each design was synthesized, implemented, and converted into a Xilinx 7-series FPGA bitstream targeting the PYNQ-Z1 platform using Vivado 2023.2. Multiple place-and-route directives were applied to each benchmark to generate implementation variants representative of realistic FPGA design flows. The resulting implementation-generated bitstreams formed the input corpus for all subsequent analyses.

2.2 Configuration-Level Audit

Each generated .bit file was parsed to isolate the FPGA configuration payload from the surrounding bitstream container. Exact byte-level comparison was then performed to identify duplicate configuration payloads and determine the effective diversity of the generated corpus. This analysis evaluates whether implementation variants correspond to genuinely distinct FPGA configurations.

2.3 Label Validation

To verify that malicious labels assigned at the RTL level remained valid after implementation, generated bitstreams were compared with their corresponding benign implementations. Cases in which synthesis optimization removed Trojan logic were identified through implementation analysis, allowing label validity to be assessed after synthesis rather than inherited directly from the RTL benchmarks.

2.4 Localization Analysis

The localization study compares trusted and Trojan-embedded bitstreams at the configuration-frame level. We first evaluate a naïve localization strategy that attributes the largest implementation differences to the Trojan. We then investigate how implementation factors, including logic density, placement provenance, routing variability, and timing closure, influence the observed configuration differences. Finally, a placement-aware verification workflow is used to compare bitstreams sharing a common placement ancestor, reducing implementation noise and exposing Trojan-induced configuration changes. Configuration bitstreams are partitioned into fixed-size comparison windows, where each window contains

eight consecutive FPGA configuration frames. For the target Xilinx PYNQ-Z1 device, this produces 1,251 comparison windows, which serve as the basic unit for byte-level localization.

3 Results and Evaluation

Table 1 summarizes the principal findings of this work. We first evaluated a naïve localization strategy based on the largest configuration differences between trusted and Trojan-inserted bitstreams. Unexpectedly, localization followed implementation density rather than Trojan location, indicating that bitstream differences are dominated by implementation characteristics instead of malicious logic.

To identify the source of this behavior, we audited the implementation-generated bitstream corpus. Configuration-level analysis revealed reduced effective dataset diversity and post-implementation label inconsistencies, demonstrating that synthesis and place-and-route alter the statistical properties of the generated bitstreams. These implementation artifacts introduce substantially more configuration changes than the Trojan itself, masking Trojan-induced modifications.

Finally, placement-aware verification was used to compare bitstreams sharing a common placement ancestor. Preserving placement provenance significantly reduced implementation noise, enabling localization to a small set of candidate configuration windows suitable for analyst inspection. These findings demonstrate that trustworthy FPGA Trojan localization depends on implementation-aware comparison rather than machine-learning detection alone.

Table 1: Summary of the principal findings.

Finding	Evidence	Impact
Dataset collapse	1383 $\rightarrow$ 431 $\rightarrow$ 140 $\rightarrow$ 13	Reduced effective diversity
Label validity	31% removed	Implementation labels require validation
Naïve localization	Density $\neq$ Trojan	Localization is unreliable
Implementation	P&R > Trojan	Trojan signal is masked noise
Placement-aware comparison	Shared placement	Enables trustworthy localization

4 Conclusion

Machine-learning detection alone is insufficient for deployment-time FPGA security because operators must determine where suspicious logic resides. This work shows that trustworthy localization depends on controlling implementation artifacts rather than simply improving detection accuracy. Through configuration auditing, label validation, and placement-aware verification, we demonstrate that preserving implementation provenance enables reliable localization of Trojan-induced changes while suppressing routing noise. These findings provide practical guidelines for future FPGA bitstream analysis and establish implementation-aware comparison as the foundation for trustworthy FPGA Trojan localization.

Acknowledgments

This work was supported by the McNair Junior Fellowship and the Office of Undergraduate Research at the University of South

Carolina. Benchmark designs are drawn from Trust-Hub, an NSF-sponsored resource; FPGA hardware and tools were provided by AMD and Xilinx through the AMD University Program.

AI-generated text disclosure: The authors used Anthropic’s Claude and Chat GPT as a drafting, editing, formatting prose, and generating figures. All technical content, experiments, and analysis were conceived and performed by the author, who reviewed and edited every section and takes full responsibility for the accuracy of the final text.

We thank Rye Stahle-Smith for his foundational contributions in evaluating the framework.

References

- [1] V. Betz and J. Rose, “Field-programmable gate array architecture,” in *Encyclopedia of Computer Science and Technology*. Springer, 2021.
- [2] K. S. Karmani and S. S. Shinde, “High-performance computing using fpgas,” *International Journal of Advanced Research in Computer and Communication Engineering*, vol. 5, no. 3, pp. 421–424, 2016. [Online]. Available: <https://researchgate.net>
- [3] A. F. Shaw, M. R. Al-Amin, and C. J. Newman, “A survey on deep learning hardware accelerators,” *ACM Computing Surveys*, vol. 57, no. 4, pp. 1–38, 2024.
- [4] Altera Corporation, “The expanding role of fpgas in edge ai,” *Electronic Design*, 2024. [Online]. Available: <https://electronicdesign.com>
- [5] S. Bhattacharya and R. S. Chakraborty, “Case of hardware trojan detection in FPGAs,” *Journal of Cybersecurity and Privacy*, vol. 4, no. 3, pp. 412–435, 2024.
- [6] S. Zeitouni and A.-R. Sadeghi, “Sok: Secure FPGA multi-tenancy in the cloud—challenges and opportunities,” in *Proceedings of the IEEE European Symposium on Security and Privacy (EuroS&P)*, 2021, pp. 452–469.
- [7] Y. Zheng, C. Zhang, S. Zhao, X. Li, and D. Feng, “Multi-tenant cloud FPGA: A survey on security, trust, and privacy,” *ACM Computing Surveys*, vol. 58, no. 2, pp. 1–39, 2026.
- [8] P. Swierczynski, M. Fyrbiak, P. Koppe, and C. Paar, “Interception of FPGA bitstreams using malicious CAD toolchains,” in *Proceedings of the International Conference on Field-Programmable Technology (FPT)*, 2022, pp. 120–127.
- [9] R. Stahle-Smith and R. Karakchi, “From bits to trust: Ai-driven reconfigurable security,” in *SoutheastCon 2026*, 2026, pp. 1–6.
- [10] R. Stahle-Smith, C. Antley, J. D. Bakos, and R. Karakchi, “Hardware-accelerated line-rate bitstream screening for secure fpga reconfiguration,” *arXiv preprint arXiv:2605.08984*, 2026.
- [11] R. Stahle-Smith and R. Karakchi, “Real-time ML-based defense against malicious payload in reconfigurable embedded systems,” in *Proceedings of the SC25 Research Posters*, 2025.
- [12] R. Stahle-Smith, C. Antley, J. D. Bakos, and R. Karakchi, “BASE: A reusable streaming preprocessing accelerator for heterogeneous FPGA systems,” 2026, manuscript/Preprint.
- [13] C. Krieg, “Reflections on trusting trusthub,” in *ICCAD*, 2023.